MATRIX MULTIPLICATION

PROGRAM 1:

Write a program to perform basic matrix operations.

```
import numpy as np
def input_matrix(rows, cols, name):
    print(f"\nEnter the elements for matrix {name} ({rows}*{cols}):")
    matrix = []
    for i in range(rows):
        row = list(map(int, input(f"Row {i+1}: ").split()))
        while len(row) != cols:
            print(f"Please enter exactly {cols} elements.")
            row = list(map(int, input(f"Row {i+1}: ").split()))
        matrix.append(row)
    return np.array(matrix)
r1=int(input("Enter no of rows for matrix A:"))
c1=int(input("Enter no of columns for matrix A:"))
r2=int(input("Enter no of rows for matrix B:"))
c2=int(input("Enter no of columns for matrix B:"))
A=input_matrix(r1,c1,'A')
B=input_matrix(r2,c2,'B')
print("\n Matrix A:")
print(A)
print("\n Matrix B:")
print(B)
if A.shape==B.shape:
    print("\n A+B(Addition):")
    print(A+B)
else:
    print("\n Addition not possible(different dimensions)")
if c1==r2:
```

```
    product=np.dot(A,B)
    print("\n A*B(Multiplication):")
    print(product)
else:
    print("\n Multiplication not possible (A's columns!=B's rows)")
    print("\n Transpose of matrix A:")
    print(np.transpose(A))
```

OUTPUT:

```
Console 2/A ×
In [2]: %runfile 'C:/Users/Windows 10/.spyder-py3/spyder.lock/prog 1.py' --wdir
Enter no of rows for matrix A: 2
Enter no of columns for matrix A:2
Enter no of rows for matrix B:2
Enter no of columns for matrix B:2

Enter the elements for matrix A (2*2):
Row 1: 1 2
Row 2: 3 4

Enter the elements for matrix B (2*2):
Row 1: 5 6
Row 2: 7 8

 Matrix A:
[[1 2]
 [3 4]]

 Matrix B:
[[5 6]
 [7 8]]
```

```
Console 2/A ×

 Matrix B:
[[5 6]
 [7 8]]

 A+B(Addition):
[[ 6  8]
 [10 12]]

 A*B(Multiplication):
[[19 22]
 [43 50]]
```

DICTIONARY OPERATION

PROGRAM 2:

Write a program to perform basic operations with dictionary and set datatypes.

```
def dictionary_operations():
    my_dict = {}
    while True:
        print("\n-- Dictionary Operations ---")
        print("1. Add key-value pair")
        print("2. Delete key")
        print("3. Search key")
        print("4. Display dictionary")
        print("5. Return to Main Menu")
        choice = input("Enter your choice: ")
        if choice == '1':
            key = input("Enter key: ")
            value = input("Enter value: ")
            my_dict[key] = value
            print(f"Added {{'{key}': '{value}'}} to dictionary.")
        elif choice == '2':
            key = input("Enter key to delete: ")
            if key in my_dict:
                del my_dict[key]
                print(f"Deleted key '{key}' from dictionary.")
            else:
                print("Key not found.")
        elif choice == '3':
            key = input("Enter key to search: ")
            if key in my_dict:
                print(f"Found: {key} => {my_dict[key]}")
            else:
```

```
            print("Key not found.")
        elif choice == '4':
            print("Dictionary Contents:", my_dict)
        elif choice == '5':
            break
        else:
            print("Invalid choice. Try again.")
def set_operations():
    my_set = set()
    while True:
        print("\n-- Set Operations ---")
        print("1. Add element")
        print("2. Remove element")
        print("3. Search element")
        print("4. Display set")
        print("5. Return to Main Menu")
        choice = input("Enter your choice: ")
        if choice == '1':
            element = input("Enter element to add: ")
            my_set.add(element)
            print(f"Added '{element}' to set.")
        elif choice == '2':
            element = input("Enter element to remove: ")
            if element in my_set:
                my_set.remove(element)
                print(f"Removed '{element}' from set.")
            else:
                print(f"Element '{element}' not found in set.")
        elif choice == '3':
            element = input("Enter element to search: ")
```

```
            if element in my_set:
                print(f"'{element}' found in set.")
            else:
                print(f"'{element}' not found in set.")
        elif choice == '4':
            print("Set Contents:", my_set)
        elif choice == '5':
            break
        else:
            print("Invalid choice. Try again.")
# Main Menu
while True:
    print("\n-- Main Menu ---")
    print("1. Dictionary Operations")
    print("2. Set Operations")
    print("3. Exit")
    main_choice = input("Enter your choice: ")
    if main_choice == '1':
        dictionary_operations()
    elif main_choice == '2':
        set_operations()
    elif main_choice == '3':
        print("Exiting Program. Goodbye!")
        break
    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

Dictionary operation:

```
In [4]: %runfile 'C:/Users/Windows 10/.spyder-py3/spyder.l

-- Main Menu ---
1. Dictionary Operations
2. Set Operations
3. Exit
Enter your choice: 1

-- Dictionary Operations ---
1. Add key-value pair
2. Delete key
3. Search key
4. Display dictionary
5. Return to Main Menu
Enter your choice: 1
Enter key: name
Enter value: Anna
Added {'name': 'Anna'} to dictionary.

-- Dictionary Operations ---
1. Add key-value pair
2. Delete key
3. Search key
4. Display dictionary
5. Return to Main Menu
```

```
Console 2/A ×
4. Display dictionary
5. Return to Main Menu
Enter your choice: 3
Enter key to search: name
Found: name => Anna

-- Dictionary Operations ---
1. Add key-value pair
2. Delete key
3. Search key
4. Display dictionary
5. Return to Main Menu
Enter your choice: 4
Dictionary Contents: {'name': 'Anna'}

-- Dictionary Operations ---
1. Add key-value pair
2. Delete key
3. Search key
4. Display dictionary
5. Return to Main Menu
Enter your choice: 2
Enter key to delete: name
Deleted key 'name' from dictionary.
```

```
Console 2/A ×
-- Dictionary Operations ---
1. Add key-value pair
2. Delete key
3. Search key
4. Display dictionary
5. Return to Main Menu
Enter your choice: 4
Dictionary Contents: {}
```

Set Operation:

```
Console 2/A ×
-- Dictionary Operations ---
1. Add key-value pair
2. Delete key
3. Search key
4. Display dictionary
5. Return to Main Menu
Enter your choice: 5

-- Main Menu ---
1. Dictionary Operations
2. Set Operations
3. Exit
Enter your choice: 2

-- Set Operations ---
1. Add element
2. Remove element
3. Search element
4. Display set
5. Return to Main Menu
Enter your choice: 1
Enter element to add: apple
Added 'apple' to set.
```

```
Console 2/A ×
Added 'apple' to set.

-- Set Operations ---
1. Add element
2. Remove element
3. Search element
4. Display set
5. Return to Main Menu
Enter your choice: 3
Enter element to search: apple
'apple' found in set.

-- Set Operations ---
1. Add element
2. Remove element
3. Search element
4. Display set
5. Return to Main Menu
Enter your choice: 4
Set Contents: {'apple'}

-- Set Operations ---
1. Add element
2. Remove element
3. Search element
4. Display set
```

```
Console 2/A ×
Set Contents: {'apple'}

-- Set Operations ---
1. Add element
2. Remove element
3. Search element
4. Display set
5. Return to Main Menu
Enter your choice: 2
Enter element to remove: apple
Removed 'apple' from set.

-- Set Operations ---
1. Add element
2. Remove element
3. Search element
4. Display set
5. Return to Main Menu
Enter your choice: 4
Set Contents: set()

-- Set Operations ---
1. Add element
2. Remove element
3. Search element
4. Display set
```

```
Console 2/A ×
2. Remove element
3. Search element
4. Display set
5. Return to Main Menu
Enter your choice: 4
Set Contents: set()

-- Set Operations ---
1. Add element
2. Remove element
3. Search element
4. Display set
5. Return to Main Menu
Enter your choice: 5

-- Main Menu ---
1. Dictionary Operations
2. Set Operations
3. Exit
Enter your choice: 3
Exiting Program. Goodbye!
```

REGEX VALIDATION

PROGRAM 3:

Write a program to implement regular expression in python.

```
import re
def menu():
    print("\n=== REGEX VALIDATION ===")
    print("1. Validate phone number")
    print("2. Validate password")
    print("3. Validate hashtag")
    print("4. Validate email")
    print("5. Exit")
def main():
    while True:
        menu()
        choice = input("Enter your choice (1-5): ")
        if choice == '1':
            phone = input("Enter phone number: ")
            pattern = r'^(?:\+91|0)?[6-9]\d{9}$'
            if re.fullmatch(pattern, phone):
                print("Valid phone number")
            else:
                print("Invalid phone number")
        elif choice == '2':
            password = input("Enter password: ")
            # At least one lowercase, one uppercase, one digit, one special char, min 8 length
            pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-
z\d@$!%*?&]{8,}$'
            if re.fullmatch(pattern, password):
                print("Strong password")
            else:
                print(" Weak password. Must include upper, lower, digit, and special symbol.")
```

```
        elif choice == '3':
            text = input("Enter text with hashtags: ")
            hashtags = re.findall(r'#\w+', text)
            if hashtags:
                print("Hashtags found:", hashtags)
            else:
                print("No hashtags found.")
        elif choice == '4':
            text = input("Enter text with email addresses: ")
            emails = re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,3}', text)
            if emails:
                print("Email addresses found:", emails)
            else:
                print(" No email address found.")
        elif choice == '5':
            print("Exiting program. Goodbye! 👏")
            break
        else:
            print("Invalid choice. Please try again!")
main()
```

OUTPUT:

```
Console 2/A ×

In [8]: %runfile 'C:/Users/Windows 10/.spyder-py3

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
Enter your choice (1-5): 1
Enter phone number: 9087654321
Valid phone number

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
Enter your choice (1-5): 1
Enter phone number: 8747993
Invalid phone number

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
```

```
Console 2/A ×

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
Enter your choice (1-5): 3
Enter text with hashtags: #inptut
Hashtags found: ['#inptut']

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
Enter your choice (1-5): input
Invalid choice. Please try again!

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
```

```
Console 2/A ×
Invalid phone number

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
Enter your choice (1-5): 2
Enter password: anna@xyz
 Weak password. Must include upper, lower, digit, and special symbol.

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
Enter your choice (1-5): 2
Enter password: Anna@12!
Strong password

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
```

```
Console 2/A ×

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
Enter your choice (1-5): 4
Enter text with email addresses: anna1234@gmail.co
Email addresses found: ['anna1234@gmail.co']

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
Enter your choice (1-5): 5
Exiting program. Goodbye! 👋
```

```
Console 2/A ×
5. Exit
Enter your choice (1-5): input
Invalid choice. Please try again!

=== REGEX VALIDATION ===
1. Validate phone number
2. Validate password
3. Validate hashtag
4. Validate email
5. Exit
Enter your choice (1-5): 4
Enter text with email addresses: xyz.gmail@.com
 No email address found.
```

FILE HANDLING OPERATIONS

PROGRAM 4:

Write a program to perform file handling operations.

```
import os
def create_file(filename):
    with open(filename, 'w') as f:
        print(f"File '{filename}' created successfully.")
def write_to_file(filename, content):
    with open(filename, 'w') as f:
        f.write(content)
    print(f"Content written to '{filename}'.")
def read_file(filename):
    if os.path.exists(filename):
        with open(filename, 'r') as f:
            content = f.read()
        print(f"Content of '{filename}':\n{content}")
    else:
        print(f"File '{filename}' doesn't exist.")
def append_to_file(filename, content):
    with open(filename, 'a') as f:
        f.write(content)
    print(f"Content appended to '{filename}'.")
def delete_file(filename):
    if os.path.exists(filename):
        os.remove(filename)
        print(f"File '{filename}' deleted successfully.")
    else:
        print(f"File '{filename}' doesn't exist.")


def main():
```

```
    filename = "sample.txt"
    while True:
        print("\n=== File Handling Operations Menu ===")
        print("1. Create File")
        print("2. Write to File")
        print("3. Read File")
        print("4. Append to File")
        print("5. Delete File")
        print("6. Exit")
        choice = input("Enter your choice (1-6): ")
        if choice == '1':
            create_file(filename)
        elif choice == '2':
            data = input("Enter content to write: ")
            write_to_file(filename, data)
        elif choice == '3':
            read_file(filename)
        elif choice == '4':
            data = input("Enter content to append: ")
            append_to_file(filename, data)
        elif choice == '5':
            delete_file(filename)
        elif choice == '6':
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid choice. Please select a valid option.")
if __name__ == "__main__":
    main()
```

OUTPUT:

```
Console 2/A ×
In [9]: %runfile 'C:/Users/Windows 10/.spyder-py3/sp

=== File Handling Operations Menu ===
1. Create File
2. Write to File
3. Read File
4. Append to File
5. Delete File
6. Exit
Enter your choice (1-6): 1
File 'sample.txt' created successfully.

=== File Handling Operations Menu ===
1. Create File
2. Write to File
3. Read File
4. Append to File
5. Delete File
6. Exit
Enter your choice (1-6): 2
Enter content to write: sample.txt
Content written to 'sample.txt'.

=== File Handling Operations Menu ===
1. Create File
```

```
Console 2/A ×
1. Create File
2. Write to File
3. Read File
4. Append to File
5. Delete File
6. Exit
Enter your choice (1-6): 3
Content of 'sample.txt':
sample.txt

=== File Handling Operations Menu ===
1. Create File
2. Write to File
3. Read File
4. Append to File
5. Delete File
6. Exit
Enter your choice (1-6): 4
Enter content to append: Machine Learning
Content appended to 'sample.txt'.

=== File Handling Operations Menu ===
1. Create File
2. Write to File
3. Read File
4. Append to File
```

```
Console 2/A ×
4. Append to File
5. Delete File
6. Exit
Enter your choice (1-6): 3
Content of 'sample.txt':
sample.txtMachine Learning

=== File Handling Operations Menu ===
1. Create File
2. Write to File
3. Read File
4. Append to File
5. Delete File
6. Exit
Enter your choice (1-6): 5
File 'sample.txt' deleted successfully.

=== File Handling Operations Menu ===
1. Create File
2. Write to File
3. Read File
4. Append to File
5. Delete File
6. Exit
Enter your choice (1-6): 3
File 'sample.txt' doesn't exist.
```

```
Console 2/A ×
2. Write to File
3. Read File
4. Append to File
5. Delete File
6. Exit
Enter your choice (1-6): 3
File 'sample.txt' doesn't exist.

=== File Handling Operations Menu ===
1. Create File
2. Write to File
3. Read File
4. Append to File
5. Delete File
6. Exit
Enter your choice (1-6): 6
Exiting program. Goodbye!
```

STUDENT DETAILS

PROGRAM 5:

Write a program to store details of the students using dataframe and perform primary operations.

```
import pandas as pd
columns = ['ID', 'Name', 'Age', 'Department']
students_df = pd.DataFrame(columns=columns)
def add_student():
    global students_df
    sid = input("Enter the Student ID: ")
    name = input("Enter Student Name: ")
    age = input("Enter Student Age: ")
    dept = input("Enter Department: ")
    new_student = pd.DataFrame([[sid, name, age, dept]], columns=columns)
    students_df = pd.concat([students_df, new_student], ignore_index=True)
    print("\n Student added successfully!\n")

def display_students():
    global students_df
    if students_df.empty:
        print("\n No student records found.\n")
    else:
        print("\n Student Records:")
        print(students_df.to_string(index=False))
        print()

def search_students():
    global students_df
    sid = input("Enter Student ID to search: ")
    result = students_df[students_df['ID'] == sid]
```

```
    if result.empty:
        print(f"\n No record found with ID: {sid}\n")
    else:
        print("\n Student found:")
        print(result.to_string(index=False))
        print()

def delete_student():
    global students_df
    sid = input("Enter Student ID to delete: ")
    if sid in students_df['ID'].values:
        students_df = students_df[students_df['ID'] != sid]
        print(f"\n Student with ID {sid} deleted successfully.\n")
    else:
        print(f"\n No student found with ID: {sid}\n")

def update_student():
    global students_df
    sid = input("Enter Student ID to update: ")
    if sid in students_df['ID'].values:
        index = students_df[students_df['ID'] == sid].index[0]
        print("\nEnter new details (press Enter to skip any field):")
        new_name = input("Enter new Name: ")
        new_age = input("Enter new Age: ")
        new_dept = input("Enter new Department: ")
        if new_name:
            students_df.at[index, 'Name'] = new_name
        if new_age:
            students_df.at[index, 'Age'] = new_age
        if new_dept:
```

```
            students_df.at[index, 'Department'] = new_dept
        print(f"\n Student with ID {sid} updated successfully.\n")
    else:
        print(f"\n No student found with ID: {sid}\n")


def main():
    while True:
        print("\n---  Student Records Menu ---")
        print("1. Add Student")
        print("2. Display All Students")
        print("3. Search Student by ID")
        print("4. Update Student Details")
        print("5. Delete Student")
        print("6. Exit")
        choice = input("Enter your choice (1-6): ")
        if choice == '1':
            add_student()
        elif choice == '2':
            display_students()
        elif choice == '3':
            search_students()
        elif choice == '4':
            update_student()
        elif choice == '5':
            delete_student()
        elif choice == '6':
            print("\n Exiting program... Goodbye!\n")
            break
        else:
            print("\n Invalid choice! Please try again.\n")
```

```
if __name__ == "__main__":
    main()
```

OUTPUT:

```
In [4]: %runfile 'C:/Users/Windows 10/.spyder-py3/untitled0.py' --wdir

--- 🎓 Student Records Menu ---
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Details
5. Delete Student
6. Exit
Enter your choice (1-6): 1
Enter the Student ID: 01
Enter Student Name: John
Enter Student Age: 34
Enter Department: MCA

 Student added successfully!


--- 🎓 Student Records Menu ---
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Details
5. Delete Student
6. Exit
Enter your choice (1-6): 1
Enter the Student ID: 02
Enter Student Name: Sarah
Enter Student Age: 45
Enter Department: MBA

 Student added successfully!
```

```
 Student added successfully!


--- 🎓 Student Records Menu ---
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Details
5. Delete Student
6. Exit
Enter your choice (1-6): 1
Enter the Student ID: 03
Enter Student Name: Adam
Enter Student Age: 40
Enter Department: MCom

 Student added successfully!


--- 🎓 Student Records Menu ---
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Details
5. Delete Student
6. Exit
Enter your choice (1-6): 2

 Student Records:
ID  Name Age Department
01  John  34        MCA
02 Sarah  45        MBA
03  Adam  40       MCom
```

```
03  Adam  40       MCom


--- 🎓 Student Records Menu ---
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Details
5. Delete Student
6. Exit
Enter your choice (1-6): 5
Enter Student ID to delete: 01

 Student with ID 01 deleted successfully.


--- 🎓 Student Records Menu ---
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Details
5. Delete Student
6. Exit
Enter your choice (1-6): 2

 Student Records:
ID  Name Age Department
02 Kevin  37        MCA
03  Adam  40       MCom
```

```
 Student Records:
ID  Name Age Department
02 Kevin  37        MCA
03  Adam  40       MCom


--- 🎓 Student Records Menu ---
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Details
5. Delete Student
6. Exit
Enter your choice (1-6): 6

👋 Exiting program... Goodbye!
```

PROCESSING TECHNIQUES

PROGRAM 6:

Write a program to implement preprocessing techniques.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler

data = {
    'Name': ['John', 'Anna', 'Mike', 'Tom', None, 'Sara'],
    'Age': [25, 28, np.nan, 22, 30, 26],
    'Salary': [50000, 60000, 52000, 58000, None, 62000],
    'Department': ['HR', 'IT', 'IT', 'Finance', 'HR', 'Finance']
}

df = pd.DataFrame(data)
print("Original Data:\n", df)

# Fill missing values
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Salary'].fillna(df['Salary'].median(), inplace=True)
df['Name'].fillna("Unknown", inplace=True)

# Label encode Department
label_encoder = LabelEncoder()
df['Department'] = label_encoder.fit_transform(df['Department'])

# Scale Salary using StandardScaler
scaler = StandardScaler()
df['Salary_Scaled'] = scaler.fit_transform(df[['Salary']])

# Normalize Age using MinMaxScaler
minmax = MinMaxScaler()
df['Age_Normalized'] = minmax.fit_transform(df[['Age']])

# Drop duplicates
df.drop_duplicates(inplace=True)

print("\nPreprocessed Data:\n", df)
```

OUTPUT:

```
In [2]: %runfile 'C:/Users/Windows 10/.spyder-py3/temp.py' --wdir
Original Data:
    Name   Age   Salary Department
0  John  25.0  50000.0         HR
1  Anna  28.0  60000.0         IT
2  Mike   NaN  52000.0         IT
3   Tom  22.0  58000.0    Finance
4  None  30.0      NaN         HR
5  Sara  26.0  62000.0    Finance

Preprocessed Data:
       Name   Age   Salary  Department  Salary_Scaled  Age_Normalized
0      John  25.0  50000.0           1      -1.561738           0.375
1      Anna  28.0  60000.0           2       0.780869           0.750
2      Mike  26.2  52000.0           2      -1.093216           0.525
3       Tom  22.0  58000.0           0       0.312348           0.000
4   Unknown  30.0  58000.0           1       0.312348           1.000
5      Sara  26.0  62000.0           0       1.249390           0.500
```